



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA
DIVISIÓN DE INGENIERÍA ELÉCTRICA
INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA
e INTERACCIÓN HUMANO COMPUTADORA



EJERCICIOS DE CLASE N° 05

NOMBRE COMPLETO: VELARDE VALENCIA JOSUE

N° de Cuenta: 320342330

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 07

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 27/09/2026

CALIFICACIÓN: _____

EJERCICIOS DE SESIÓN:

1. Actividades realizadas. Una descripción de los ejercicios y capturas de pantalla de bloques de código generados y de ejecución del programa

Para el primer ejercicio se separaron las piezas del rover desde Blender con la guía del profesor en clase, solo se tuvieron que separar las llantas porque con clase pudimos separar el brazo y sus partes.

Agregamos la lista de los modelos de la siguiente manera a Visual Studio:

```
//Lista de Modelos a importar
Model roverbace;
Model brazobase;
Model brazouno;
Model brazodos;
Model pinza;
Model frontalmasz;
Model frontalmenosz;
Model mediomasz;
Model mediomenosz;
Model traseramasz;
Model traseramenosz;
```

Y las importamos con LoadModel();

```
roverbace = Model();
roverbace.LoadModel("Models/roverbace.obj");
brazobase = Model();
brazobase.LoadModel("Models/brazobase.obj");
brazouno = Model();
brazouno.LoadModel("Models/brazouno.obj");
brazodos = Model();
brazodos.LoadModel("Models/brazodos.obj");
pinza = Model();
pinza.LoadModel("Models/pinza.obj");
frontalmasz = Model();
frontalmasz.LoadModel("Models/frontal+z.obj");
frontalmenosz = Model();
frontalmenosz.LoadModel("Models/frontal-z.obj");
mediomasz = Model();
mediomasz.LoadModel("Models/medio+z.obj");
mediomenosz = Model();
mediomenosz.LoadModel("Models/medio-z.obj");
traseramasz = Model();
traseramasz.LoadModel("Models/trasera+z.obj");
traseramenosz = Model();
traseramenosz.LoadModel("Models/trasera-z.obj");
```

Posteriormente armamos el modelo del rover con cada pieza, empezando por el brazo que toma su origen de la base.

```

//Base Brazo
model = modelaux;
color = glm::vec3(1.0f, 0.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
model = glm::translate(model, glm::vec3(-2.5f, 1.5f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
brazobase.RenderModel();
//Tubo 1
color = glm::vec3(0.5f, 0.5f, 0.5f);
model = glm::translate(model, glm::vec3(0.8f, 0.8f, 0.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
brazouno.RenderModel();
//Tubo 2
color = glm::vec3(0.0f, 0.0f, 1.0f);
model = glm::translate(model, glm::vec3(2.5f, 2.5f, 0.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion2()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
brazodos.RenderModel();
//Pinza
color = glm::vec3(1.0f, 0.0f, 1.0f);
model = glm::translate(model, glm::vec3(-2.5f, 3.0f, 0.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion3()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
pinza.RenderModel();

```

Organizamos la jerarquía del brazo partiendo del origen para que todos los elementos del brazo giren de acuerdo a sus antecesores, las rotaciones son simples pueden dar giros de 360 grados en cada punto. A continuación se dibujan las llantas delanteras.

```

//Llanta frontal +Z
model = modelaux;
color = glm::vec3(0.5f, 0.0f, 0.5f);
model = glm::translate(model, glm::vec3(-2.2f, 1.2f, 3.2f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion4()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion4()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
frontalmasz.RenderModel();

//Llanta frontal -Z
model = modelaux;
color = glm::vec3(0.0f, 0.0f, 1.0f);
model = glm::translate(model, glm::vec3(-2.2f, 1.2f, -3.2f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion5()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion5()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
frontalmenosz.RenderModel();

```

Usamos los dos modelos diferentes de las llantas delanteras y les asignamos una condición en Window.cpp que limita la articulación a 45 grados tanto arriba como abajo desde el punto de origen de la pieza, el origen del modelo se encuentra en la parte superior que se une a la base del rover, pudiendo girar hacia arriba y hacia abajo.

```

if (key == GLFW_KEY_J)
{
    theWindow->articulacion4 += 10.0f;

    if (theWindow->articulacion4 > 45.0f)
    {
        theWindow->articulacion4 = 45.0f;
    }
}
if (key == GLFW_KEY_K)
{
    theWindow->articulacion4 -= 10.0f;

    if (theWindow->articulacion4 < -45.0f)
    {
        theWindow->articulacion4 = -45.0f;
    }
}

```

De esta manera limitamos la articulación en la definición de las teclas.

```

//Llanta media derecha
model = modelaux;
color = glm::vec3(1.0f, 0.5f, 0.0f);
model = glm::translate(model, glm::vec3(2.2f, 0.5f, 3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion6()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion6()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
mediomasz.RenderModel();

// Llanta media izquierda
model = modelaux;
color = glm::vec3(0.5f, 0.5f, 1.0f);
model = glm::translate(model, glm::vec3(2.3f, 0.5f, -3.1f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion7()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion7()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
mediomenosz.RenderModel();

```

Continuando con el modelo, agregamos las llantas del medio con la misma lógica que las frontales, acomodándolas en la parte lateral del rover, donde hay un cuadro que hace que encajen unos pequeños ejes (ahí se encuentra el origen del modelo).

```

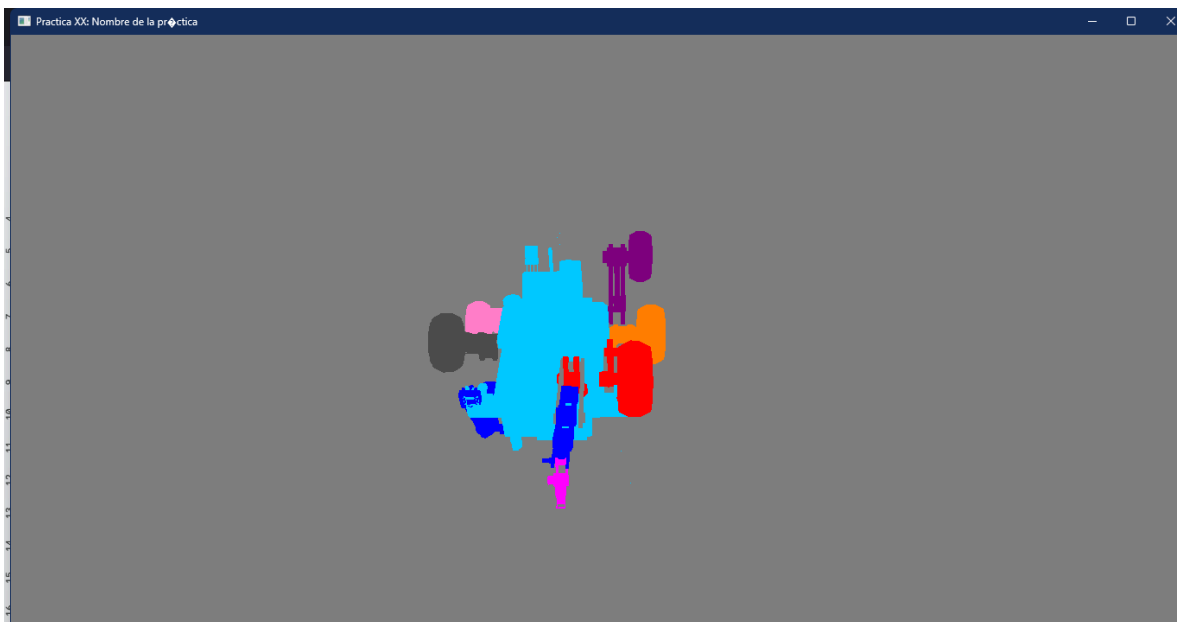
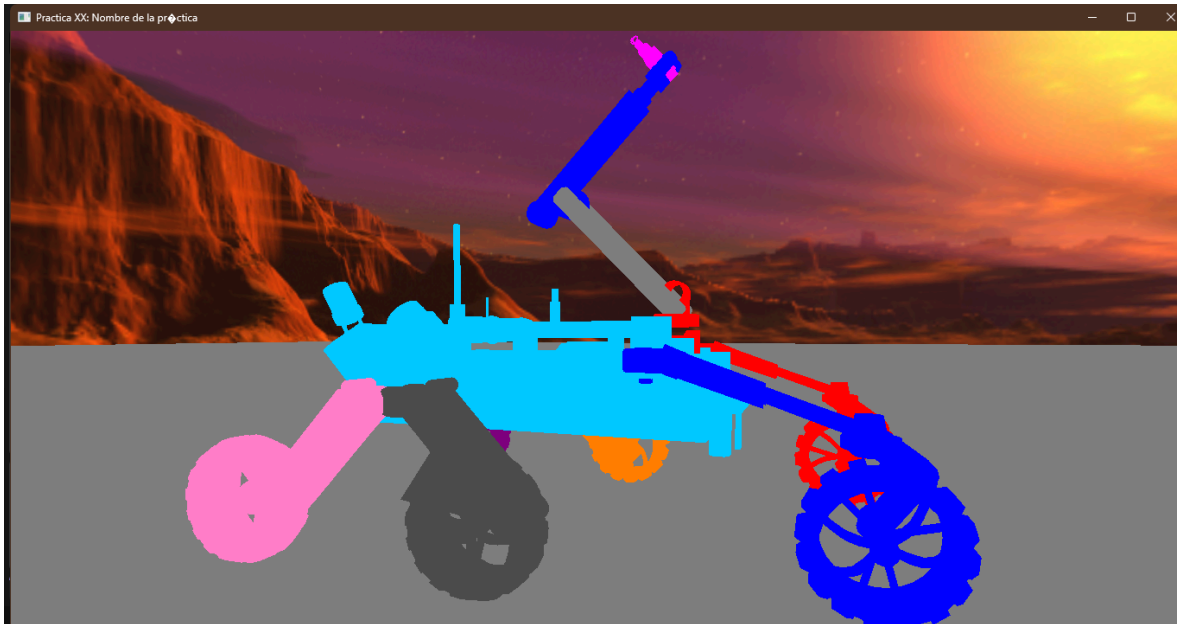
//Llanta trasera derecha
model = modelaux;
color = glm::vec3(0.5f, 0.0f, 0.5f);
model = glm::translate(model, glm::vec3(4.0f, 0.5f, 3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion8()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion8()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
traseramasz.RenderModel();

```

```
// Llanta trasera izquierda
model = modelaux;
color = glm::vec3(1.0f, 0.0f, 1.0f);
model = glm::translate(model, glm::vec3(4.0f, 0.5f, -3.1f));
model = glm::translate(model, glm::radians(mainWindow.getarticulacion9()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion9()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
traseramenosZ.RenderModel();
```

Aplicamos la misma lógica a las llantas traseras.

La ejecución final se muestra a continuación:



2. Problemas presentados.

No se presentó ningún problema durante la realización de la práctica.

3. Conclusión:

Gracias al desarrollo de esta práctica comprendí de mejor manera las bases del modelado jerárquico y del modelado y separado de partes de modelos en Blender, la explicación me pareció muy buena, clara y concisa.